

Starvation, Priority Inversion, and Feedback Scheduling

Week 6: Progress guarantees, resource protocols, MLQ, and MLFQ

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. connect the Week 5 algorithms to progress and fairness requirements;
2. distinguish starvation, inversion, deadlock, livelock, and blocking;
3. formulate aging and conditional waiting-time bounds;
4. explain transitive inheritance and ceiling-based blocking bounds;
5. trace strict and weighted MLQ with explicit queue rules;
6. trace MLFQ quantum/allotment accounting, demotion, and boosts;
7. analyze response times with bounded blocking; and
8. connect these mechanisms to modern kernel and group scheduling.

Guiding question

How can a scheduler express importance without sacrificing isolation, fairness, or bounded latency?

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Week 5 bridge: choosing a job is only the beginning

Already covered	Selection principle	Question for this week
FCFS	oldest ready job	how large can convoy delays become?
SJF	shortest available burst	can a long job be postponed forever?
SRTF	least remaining service	does a good mean imply fairness?
Priority (both variants)	assigned importance	what if an important job needs a low-priority owner?
RR	bounded turns among peers	who allocates service between classes?

Notation for the extension

Here larger π means higher priority; equivalently $\pi = -p$ for Week 5's smaller-is-higher rank p . In queue diagrams, Q_0 is highest.

The introductory policies and their worked priority schedules remain in Week 5.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service**
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Progress failures: diagnose the state before choosing a fix

Problem	What is happening?	Relevant mechanism
CPU starvation	ready, repeatedly not selected	aging, fair admission, guaranteed service
Priority inversion	blocked on a lower-priority owner	inheritance or ceiling protocol
Deadlock	an unresolvable dependency wait	lock ordering, prevention, recovery
Livelock	active retries, no useful completion	coordinated arbitration or backoff
Ordinary blocking	waiting for a required event	bound/measure event latency
Lock convoy	many tasks serialize behind a holder	shorter locks, less contention

Safety is not progress

Mutual exclusion can hold while every task waits forever. Backoff or inheritance alone does not prove starvation freedom or deadlock freedom.

Starvation: service accounting and a counterexample

Define cumulative CPU service since release:

$$x_i(t) = \int_{a_i}^t \mathbf{1}\{\text{CPU executes } i \text{ at } u\} du.$$

A job continuously ready after t_0 receives no further service if $x_i(t) = x_i(t_0)$ for all $t \geq t_0$.

An infinite workload, not just a long finite queue

One CPU, zero overhead, strict priority. Jobs (a, b, π) :

$$L = (0, 2, 1), \quad H_k = (k, 1, 3), \quad k = 0, 1, 2, \dots$$

Each H_k runs on $[k, k+1)$. At every completion another high-priority job is ready, so $x_L(t) = 0$ forever.

A finite prefix demonstrates a long wait, not indefinite starvation. Spare average capacity alone does not bound delays from arbitrarily large bursts.

Aging: an explicit priority-update rule

Larger effective priority means more urgent:

$$\pi_i^{\text{eff}}(t) = \min\left(\pi_{\max}, \pi_i^{\text{base}} + \delta \left\lfloor \frac{w_i(t)}{\Delta} \right\rfloor\right), \quad \delta, \Delta > 0.$$

$w_i(t)$ accumulates ready waiting since the most recent dispatch; this reference policy resets credit at dispatch and does not age blocked time.

Specify the mechanism

Add δ every Δ waiting units; cap at $\pi_{\max}$; schedule threshold events; define fair ties and whether boosts preempt.

Continuous aging, retained credit, and periodic global boosts are alternatives, not interchangeable specifications.

Limits and trade-offs

Queue maintenance costs time. Frequent resets can oscillate priorities. A cap below a dominating class still permits starvation.

Aging: from rank improvement to a waiting bound

Suppose competing effective ranks never exceed $\pi_H \leq \pi_{\max}$:

$$t_{\text{rank}} = \Delta \left\lceil \frac{\max(0, \pi_H - \pi_L)}{\delta} \right\rceil.$$

A conditional first-dispatch guarantee

At this rank, suppose at most m peers precede the job, each can execute at most q before it, residual non-preemptible execution is at most d , and new arrivals cannot overtake:

$$W_{\text{first}} \leq t_{\text{rank}} + d + mq.$$

Example: $\pi_L = 1$, $\pi_H = 5$, $\delta = 1$, $\Delta = 2$, $m = 2$, $q = 1$, $d = 1$ gives $t_{\text{rank}} = 8$ and $W_{\text{first}} \leq 11$ time units.

A service guarantee is stronger than a priority boost

A class might receive, in every continuously backlogged interval of length ℓ ,

$$\text{service}(\ell) \geq \beta(\ell) = \rho \max(0, \ell - \theta), \quad 0 < \rho \leq 1.$$

ρ is a minimum service rate; θ is a latency allowance.

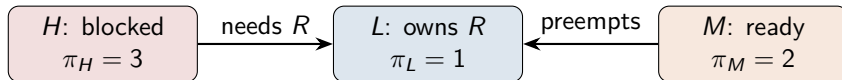
- A job alone in the class, needing b service, completes within $\theta + b/\rho$ under this guarantee.
- Multiple jobs still need internal fairness and bounded work ahead.
- Admission control limits demand; backpressure limits queue growth.
- A weight without a bound on competitors, or a quota that only caps usage, is not this lower-service guarantee.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols**
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Priority inversion: the scheduler follows a dependency

H is blocked because L owns its required mutex. Unrelated medium-priority work M can preempt L and extend the delay.



Decompose the blocked interval

B_H = remaining owner execution + intervening execution + suspension/overhead. PIP targets unrelated medium-priority interference, not every term.

CPU aging cannot make a mutex available. Accelerate the dependency's owner, not just the blocked requester.

Priority inheritance protocol (PIP)

When H blocks on a mutex owned by L :

1. raise L to the highest effective priority among its blocked waiters;
2. propagate donation through nested lock dependencies;
3. recompute effective priority as waiters and owned locks change.

$$\pi_i^{\text{eff}} = \max\left(\pi_i^{\text{base}}, \max_{j \in \mathcal{W}(i)} \pi_j^{\text{eff}}\right),$$

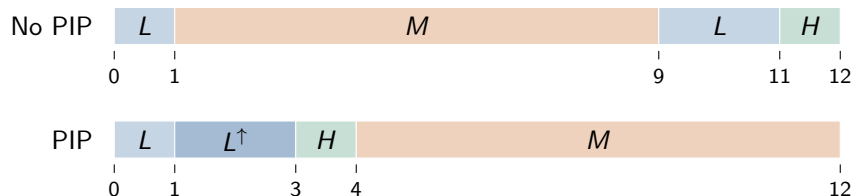
where $\mathcal{W}(i)$ contains waiters on all PI-enabled mutexes held by i ; omit the inner maximum if empty.

Inheritance changes rank, not ownership or readiness

H remains blocked. A runnable owner outranks unrelated medium-priority work, but still-higher work can interfere. Basic PIP does not prevent deadlock or all chained blocking.

PIP example 1: measure inversion before and after

Jobs (a, b, π) : $L = (0, 3, 1)$, $M = (1, 8, 2)$, $H = (1, 1, 3)$. L holds R throughout its 3 CPU units; H requests R on arrival. Lock operations and switches cost zero; M never uses R .

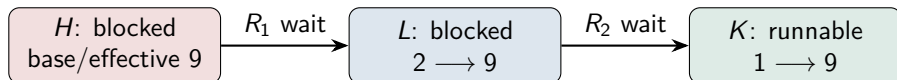


$$B_H^{\text{no PIP}} = 11 - 1 = 10 = 2 + 8, \quad B_H^{\text{PIP}} = 3 - 1 = 2.$$

$L^\uparrow$ inherits H 's priority; H stays blocked until R is released.

PIP example 2: transitive donation across two locks

Snapshot: $\pi_H = 9$, $\pi_M = 5$, $\pi_L = 2$, $\pi_K = 1$. H waits on R_1 owned by L ; L waits on R_2 owned by K . M is independent and ready; no other tasks, arrivals, or overhead.



If K needs 2 CPU units to release R_2 and then L needs 1 to release R_1 , H 's remaining blocking is $2 + 1 = 3$ units with PIP.

Correct restoration is also transitive

Recompute from all held locks and remaining waiters after unlock, timeout, or cancellation. Do not blindly reset to base priority.

Priority ceilings: specify which protocol

Resource ceiling: $\Pi(R) = \max\{\pi_i : i \text{ may lock } R\}$.

Original ceiling protocol

A free lock can be acquired only if the requester's effective rank exceeds all ceilings of locks held by *other* tasks. Inherit priority when this rule blocks a requester.

Immediate ceiling protocol

At acquisition, immediately raise the owner's effective rank to at least the resource ceiling, even without a waiting high-priority task.

- Classical guarantees assume one CPU, fixed base ranks, correct ceilings, nested locks, no equal-rank preemption, and no suspension while holding locks.
- These assumptions are stronger than “use a high-priority mutex.”

Deriving a ceiling-based blocking bound

Under the classical ceiling-protocol assumptions, $C_{k,R}^{\text{CS}}$ bounds the CPU duration of a lower-priority task's critical section:

$$B_i \leq \max \left(\{0\} \cup \{C_{k,R}^{\text{CS}} : \pi_k < \pi_i, \Pi(R) \geq \pi_i\} \right).$$

Example: filter the resources before taking a maximum

Target $\pi_i = 4$. A lower-priority task uses R_A with ceiling 5 for 2 ms; another uses R_B with ceiling 3 for 7 ms. Only R_A qualifies: $B_i \leq 2$ ms, not 7 ms and not 9 ms.

- Include nested execution in the enclosing critical-section duration.
- Add higher-priority interference separately in response-time analysis.
- Basic PIP need not yield this single-section maximum: multiple blocking episodes can occur.

Choose the remedy for the actual dependency

Mechanism	Helps with	Does not automatically solve
Aging / global boost	low-priority ready waiting	unavailable locks or I/O
PIP	medium-priority interference	cycles, suspension, overload
Ceiling protocol	bounded lower-priority blocking	arbitrary multicore locking
Global lock order	circular-wait deadlock	inversion or unfair service
Shorter / split locks	serialization and convoys	every fairness requirement
Admission / backpressure	excessive offered work	correctness of lock acquisition

Timeouts need rollback or cancellation semantics. Blind retries can turn deadlock into livelock rather than useful progress.

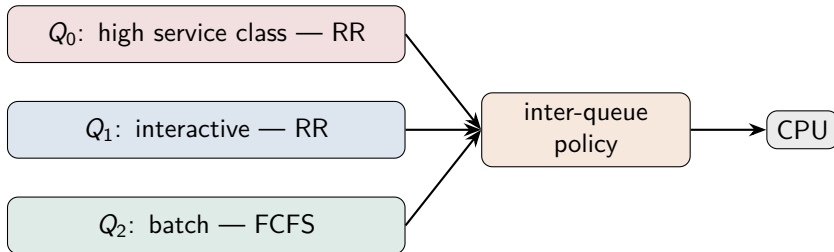
Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling**
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Multilevel queue (MLQ): static classification

Definition

Runnable entities are permanently assigned to classes/queues. Each queue has an internal policy, and a separate policy allocates CPU time among queues.



Static classification is useful when classes have genuine semantic differences, not merely inferred behavior.

Two independent policy layers

Within each queue

- FIFO, RR, priority, or deadline;
- queue-specific quantum;
- class-specific admission control.

Between queues

- strict fixed priority;
- weighted time shares;
- reservations or bandwidth caps;
- hierarchical fair sharing.

Common mistake

Saying “MLQ uses RR” is incomplete. RR may govern one queue, while another policy decides when that queue runs at all.

Example conventions: state the entire queue policy

All following MLQ/MLFQ examples use one CPU, one burst per job, no I/O, zero overhead, and time starting at 0. Intervals are $[s, e)$.

$$T_i = c_i - a_i, \quad W_i = T_i - b_i, \quad R_i = s_i - a_i.$$

s_i is first start and c_i completion. Report time-unit means; R_i here is first-service delay, not the later real-time response bound.

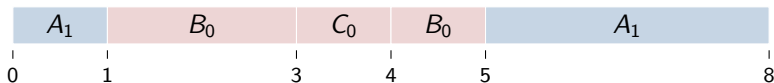
Strict MLQ examples

Q_0 outranks Q_1 , which outranks Q_2 if present. Higher-queue arrivals preempt; equal-queue arrivals do not. Preserve a preempted job's unused slice and place it at its queue's front.

- Process completion before any requeue; an exhausted job leaves the system.
- Enqueue arrivals before quantum-expired jobs; simultaneous arrivals use alphabetical order.
- Queue membership is fixed for MLQ. Labels A_k mean job A executes in Q_k .

Strict MLQ example 1: three jobs in two queues

Workload (a, b, Q) : $A = (0, 4, Q_1)$, $B = (1, 3, Q_0)$, $C = (2, 1, Q_0)$. Q_0 : RR with $q = 2$; Q_1 : FCFS.
Strict inter-queue priority; zero overhead.



Job	c_i	T_i	W_i	R_i
A	8	8	4	0
B	5	4	1	0
C	4	2	1	1
Mean		4.67	2.00	0.33

FCFS within Q_1 does not prevent preemption by Q_0 . C waits for B 's slice to end.

Strict MLQ example 2: six jobs and three queues

Workload (a, b, Q) :

$$A = (0, 6, Q_2), \quad B = (0, 4, Q_1), \quad C = (2, 3, Q_0), \\ D = (3, 2, Q_0), \quad E = (6, 2, Q_1), \quad F = (8, 1, Q_0).$$

Q_0 : RR $q = 2$; Q_1 : RR $q = 3$; Q_2 : FCFS. Strict queue priority; preserve slices across higher-queue interruptions; zero overhead.



Final means (time units; rounded)

$$\overline{T} = 7.33 \quad \overline{W} = 4.33 \quad \overline{R} = 2.67$$

Strict priority versus weighted service

	Strict inter-queue priority	Weighted allocation
rule	serve highest nonempty queue	give queue k a configured share $w_k / \sum_j w_j$
strength	low latency for high class	protects lower-class bandwidth
failure mode	lower queues may starve	latency depends on allocation period and burstiness
suitable for	truly dominant critical class	tenants, users, service groups

Unused capacity

A work-conserving weighted scheduler redistributes unused capacity among active queues. State whether unused credit is discarded or carried forward; weights alone do not specify that rule.

Weighted MLQ: one explicit cyclic-budget policy

Two fixed queues, FCFS within each. Repeatedly visit Q_0 for up to 3 CPU units, then Q_1 for up to 1 unit. Start in Q_0 at $t = 0$.

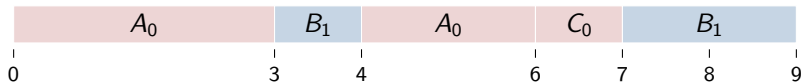
- A budget belongs to the queue, not one job; a completion passes residual budget to the next FCFS job there.
- On budget expiry, switch queues; an unfinished job retains the head position.
- If a queue empties, forfeit its residual budget and immediately visit the other. Empty visits consume no time; there is no banked credit.
- Arrivals do not interrupt an active visit; enqueue them before selection at a boundary.

with both queues backlogged: $\rho_0 = \frac{3}{4}, \quad \rho_1 = \frac{1}{4}.$

This is a pedagogical weighted-MLQ variant, not an implementation claim about Linux. Short finite workloads need not receive the long-run 3:1 ratio.

Weighted MLQ example 1: shared queue budgets

Workload (a, b, Q) : $A = (0, 5, Q_0)$, $B = (0, 3, Q_1)$, $C = (2, 1, Q_0)$. Cyclic budgets $(3, 1)$; FCFS within queues; start Q_0 ; zero overhead.



Job	c_i	T_i	W_i	R_i
A	6	6	1	0
B	9	9	6	3
C	7	5	4	4
Mean		6.67	3.67	2.33

At $t = 6$, C uses Q_0 's remaining budget. At $t = 8$, an empty Q_0 is skipped, so B continues.

Weighted MLQ example 2: six jobs and unused capacity

Workload (a, b, Q) :

$$A = (0, 6, Q_0), \quad B = (0, 4, Q_1), \quad C = (2, 3, Q_0),$$

$$D = (4, 2, Q_1), \quad E = (6, 2, Q_0), \quad F = (9, 1, Q_1).$$

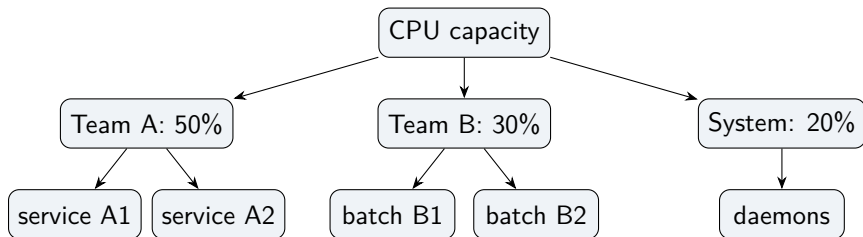
Cyclic budgets $(3, 1)$; FCFS within queues; start Q_0 ; discard unused visit budget, skip empty queues; zero overhead.



Final means (time units; rounded)

$$\overline{T} = 10.17 \quad \overline{W} = 7.17 \quad \overline{R} = 5.67$$

Hierarchical scheduling expresses organizational policy



- First divide capacity among groups; then schedule tasks inside each group.
- This prevents a user with many runnable threads from automatically receiving a larger organizational share.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling**
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

MLFQ: infer behavior from observed execution

Core idea

MLFQ dynamically changes queue level using recent CPU consumption and waiting history, approximating short-job preference without knowing future bursts.

- New jobs enter the highest queue; behavior can preserve or change their level. Short latency depends on competing work.
- Tasks that repeatedly consume full allotments move downward.
- Lower queues use longer quanta, reducing overhead for CPU-bound work.
- Periodic boosts or aging restore opportunity and adapt to phase changes.

MLFQ is a design family, not one algorithm

Queue count, quanta, allotments, boost interval, I/O accounting, and tie rules must all be specified.

A complete reference rule set

One pedagogical MLFQ can be defined by:

1. run the highest nonempty queue; preempt when a higher queue becomes ready;
2. use RR within each queue;
3. admit new jobs to Q_0 ;
4. charge all CPU time at a level against an *allotment*, even across early yields;
5. demote after the allotment is exhausted; and
6. every B time units, boost all runnable jobs, including the running job, to Q_0 and reset their quantum/allotment budgets.

Why allotment rather than one uninterrupted quantum?

Accumulated accounting prevents a CPU-bound task from avoiding demotion by yielding just before each timer interrupt.

MLFQ accounting: three quantities, not one timer

For each job maintain remaining service r_i , remaining slice s_i , and remaining level allotment a_i^{level} .

$$(r_i, s_i, a_i^{\text{level}}) \leftarrow (r_i - d, s_i - d, a_i^{\text{level}} - d) \quad \text{after } d \text{ CPU units.}$$

- $r_i = 0$: finish, even if the quantum or allotment also expires.
- Allotment exhausted: demote, initialize the next level's budgets.
- Slice exhausted only: requeue at the same level with a fresh slice.
- Higher-level preemption: preserve both budgets; resume from the front of the interrupted queue. Equal-level arrivals do not preempt.
- At the bottom level, use RR with no further demotion.

a_i^{level} is a budget, not arrival time a_i . Both examples explicitly specify whether an allotment spans one or two quanta.

MLFQ event order makes the trace reproducible

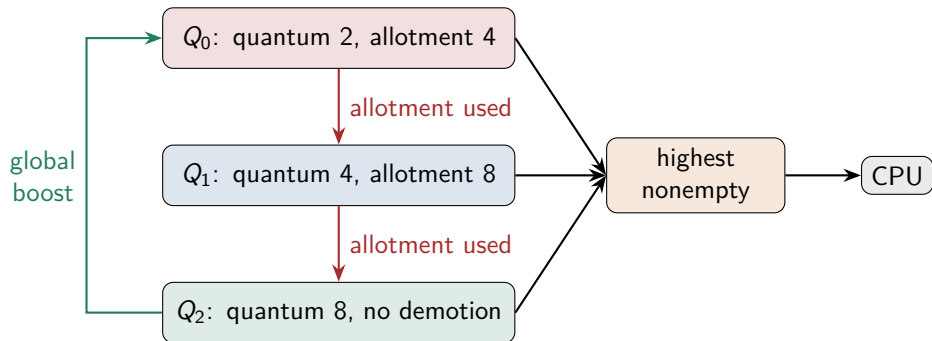
At a shared event time t :

1. Charge elapsed CPU service; remove a completed job first.
2. Determine any demotion or expired-slice requeue, but hold that enqueue temporarily.
3. Enqueue arrivals at Q_0 in alphabetical order; then append the expired/demoted job to its resulting queue.
4. If t is a boost time, gather all unfinished runnable jobs, including the current one, order by original arrival then name, and reset them into Q_0 .
5. Apply higher-queue preemption if needed, then select the highest queue's head.

This is one fully specified teaching policy

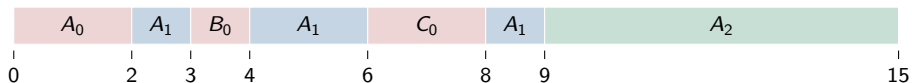
Other merge orders are possible. Our finite examples have no I/O; with blocking, separately specify wake-up placement, preserved usage, and how missed boosts are applied.

MLFQ structure and feedback



MLFQ example 1: three jobs and preserved usage

Workload (a, b) : $A = (0, 12)$, $B = (3, 1)$, $C = (6, 2)$. Quanta $(2, 4, 8)$; allotments $(2, 4, \infty)$; no boosts. All enter Q_0 ; preserve slice/allotment on higher-queue preemption; zero overhead.

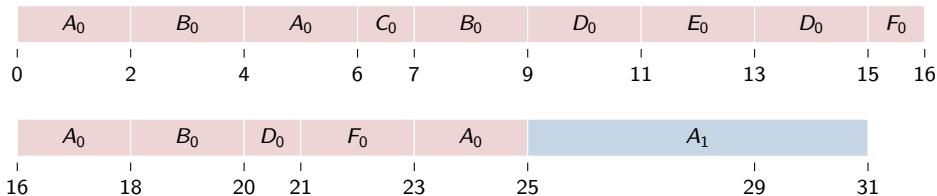


Job	c_i	T_i	W_i	R_i
A	15	15	3	0
B	4	1	0	0
C	8	2	0	0
Mean		6.00	1.00	0.00

A consumes $1 + 2 + 1 = 4$ units in Q_1 across interruptions, so it demotes at $t = 9$, not after a fresh four-unit slice.

MLFQ example 2: six jobs, two-quanta allotments, and a boost

Workload (a, b) : $A = (0, 14)$, $B = (1, 6)$, $C = (4, 1)$, $D = (6, 5)$, $E = (11, 2)$, $F = (14, 3)$. Quanta $(2, 4, 8)$; allotments $(4, 8, \infty)$; boost every 16 units; zero overhead.



Two rows continue one timeline. At $t = 16$, boost order is A, B, D, F with remaining service 10, 2, 1, 2; all budgets reset, including running F 's. Subscript = queue.

Final means (time units; rounded)

$$\overline{T} = 13.17 \quad \overline{W} = 8.00 \quad \overline{R} = 1.17$$

Response and throughput trade-offs in MLFQ

Short upper-level quanta

- improve response to new tasks;
- increase timer/context-switch cost;
- may disrupt cache locality.

Long lower-level quanta

- amortize dispatch overhead;
- favor throughput and locality;
- raise latency if boosts are too rare.

Parameter interaction

Queue quanta cannot be tuned independently of allotments, boost interval, arrival rate, and context-switch cost.

Gaming, phase changes, and periodic boosts

- **Gaming:** a task yields before its quantum expires to retain high priority.
- **Countermeasure:** charge cumulative CPU usage per level, not only uninterrupted bursts.
- **Phase change:** a formerly CPU-bound task becomes interactive but remains in a low queue.
- **Countermeasure:** periodic global boost or wait-time promotion.
- **Boost too often:** CPU-bound jobs repeatedly regain top priority.
- **Boost too rarely:** lower-level response becomes poor and starvation-like.

Security perspective

Any behavior-based policy is an incentive system. Ask how an adversarial or merely unusual workload can manipulate its signals.

MLQ versus MLFQ

	MLQ	MLFQ
queue membership	static semantic class	dynamic behavioral estimate
movement	normally none	demotion/promotion/boost
primary use	isolation among known classes	responsiveness without burst knowledge
starvation	possible under strict inter-queue priority	possible if boost/aging is absent or inadequate
predictability	clearer class contract	workload- and parameter-dependent
gaming surface	class assignment	yield/usage patterns

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis**
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 Synthesis

Priority is not the same as a deadline guarantee

A high priority expresses ordering; a guarantee requires a workload model and schedulability analysis.

- Periodic/sporadic task $\tau_i = (C_i, T_i, D_i)$: worst-case execution C_i , period/minimum interarrival T_i , relative deadline D_i .
- Fixed-priority preemptive scheduling often assigns priority by period or deadline.
- Blocking, release jitter, interrupts, and overhead must be included for credible guarantees.

General-purpose “real-time priority” is dangerous

A runaway highest-priority thread can prevent essential lower-priority system work. Privilege, admission control, watchdogs, and budgets matter.

Fixed-priority response-time analysis

One preemptive CPU; sporadic tasks with constrained deadlines $D_i \leq T_i$; fixed priorities; valid blocking bounds; no release jitter, self-suspension, or overhead:

$$R_i^{(k+1)} = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j,$$

where $hp(i)$ is the set of higher-priority tasks and B_i bounds lower-priority resource blocking.

1. initialize $R_i^{(0)} = C_i + B_i$;
2. iterate until a fixed point or until $R_i > D_i$;
3. declare the task schedulable if the fixed point satisfies $R_i \leq D_i$.

Interpretation

The ceiling counts how many jobs of each higher-priority task can interfere during τ_i 's response window.

Here R_i means release-to-completion response, not first-service delay in the finite-job examples. A converged upper bound within D_i establishes schedulability.

Response-time example 1: three tasks and one blocking term

Full task set (C, T, D) , in descending priority: $\tau_1 = (1, 4, 4)$, $\tau_2 = (2, 7, 7)$, $\tau_3 = (1, 20, 20)$.
Protocol-derived blocking bounds: $(B_1, B_2, B_3) = (0, 1, 0)$.

$$R_2^{(0)} = C_2 + B_2 = 3,$$

$$R_2^{(1)} = 3 + \left\lceil \frac{3}{4} \right\rceil (1) = 4,$$

$$R_2^{(2)} = 3 + \left\lceil \frac{4}{4} \right\rceil (1) = 4.$$

The bound converges at $R_2 = 4 \leq D_2 = 7$. Also $R_1 = 1$, $R_3 = 4$; all three meet their deadlines under this model.

Why include B_2 ?

Even a high-priority task can be delayed once by a lower-priority critical section under a bounded locking protocol.

Response-time example 2: five tasks and repeated interference

Tasks are ordered by decreasing priority; $D_i = T_i$. Blocking bounds B_i are assumed justified by the resource protocol. No jitter, suspension, or overhead.

Task	C_i	$T_i = D_i$	B_i
τ_1	1	5	0
τ_2	1	8	0
τ_3	2	12	1
τ_4	3	20	1
τ_5	4	40	0

$$R_5^{(k)} : 4 \rightarrow 11 \rightarrow 14 \rightarrow 16 \rightarrow 17 \rightarrow 18 \rightarrow 18.$$

Final bounds: $(R_1, R_2, R_3, R_4, R_5) = (1, 2, 5, 10, 18)$; every $R_i \leq D_i$.

Crossing an arrival threshold adds an interfering job; iterate until the bound truly stabilizes. These are response bounds, not means of a finite Gantt chart.

Rate-monotonic utilization bound: sufficient, not necessary

For n independent periodic tasks with $D_i = T_i$, fixed priorities by shorter period, and no blocking/overhead:

$$\sum_{i=1}^n \frac{C_i}{T_i} \leq n \left(2^{1/n} - 1 \right)$$

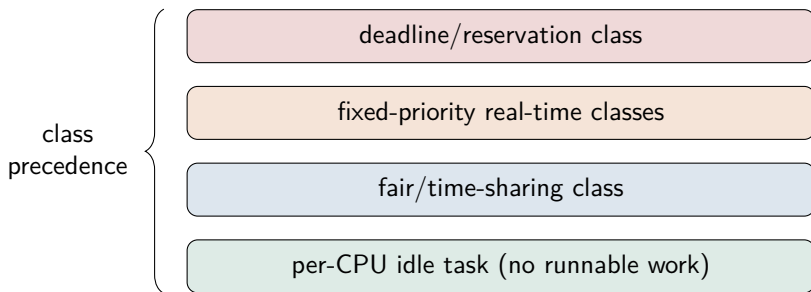
is a sufficient schedulability test.

- The bound approaches $\ln 2 \approx 0.693$ as $n \rightarrow \infty$.
- Failing the bound does *not* prove failure; exact response-time analysis may still pass.
- Harmonic periods can schedule at much higher utilization, potentially up to 100% under the ideal model.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures**
- 8 Implementation and Verification
- 9 Synthesis

Production kernels combine scheduling classes



- Each class defines its own ready structures and selection rule.
- The kernel first selects the highest eligible class, then a task within it.
- Configured admission/bandwidth controls can limit high-class demand; this is not an unconditional starvation guarantee.

Conceptual ordinary Linux class ordering; special kernel/configuration paths are omitted. Background user jobs are not the per-CPU idle task.

Linux fair scheduling: from CFS toward EEVDF

- CFS modeled weighted fair sharing using per-entity virtual runtime and historically selected the least-served entity from an ordered run queue.
- Current Linux fair-class development uses **EEVDF**: eligible entities are ordered by virtual deadlines, combining lag-based fairness with latency requests.
- Nice values remain weights, not strict fixed-priority levels; higher weight means a larger long-run CPU share among competing fair tasks.
- Separate Linux classes handle deadline and POSIX real-time policies; cgroups add hierarchical shares and bandwidth controls.

Modern correction

Describing Linux simply as “CFS chooses the leftmost red-black-tree node” is now historically useful but incomplete for the evolving fair class.

Source: Linux kernel EEVDF scheduler documentation; checked September 16, 2026.

Windows: preemptive priorities with dynamic boosts

- Windows schedules threads at priorities 0–31; a ready higher-priority thread preempts a lower-priority thread.
- Equal-priority ready threads receive round-robin time slices.
- Process priority class and thread-relative priority determine base priority.
- Variable-priority threads may receive temporary boosts after relevant waits; real-time-class threads require care and do not use ordinary dynamic boosts.

Priority is broader than CPU time

Background work can still harm responsiveness through memory, storage, and lock contention; modern systems also expose resource and quality-of-service controls.

Source: Microsoft Learn, Scheduling Priorities; checked September 16, 2026.

Containers and groups need hierarchical control

Per-thread fairness alone does not isolate tenants.

- **Weight/share:** divide spare CPU proportionally among active groups.
- **Quota/bandwidth:** cap CPU consumption over a period.
- **Affinity/cpuset:** restrict eligible processors and NUMA placement.
- **Utilization hints:** guide placement/frequency decisions; they are not reservations of CPU time.

Containerization insight

Containers do not contain a separate CPU scheduler by default; the host kernel schedules their threads under cgroup and namespace policy.

Heterogeneous multicore adds a placement problem

Scheduling now chooses both *when* and *where* a task runs.

Performance concerns

- CPU capacity and microarchitecture;
- cache/NUMA locality;
- migration and warmup cost;
- deadline or frame latency.

Energy/thermal concerns

- performance per watt;
- frequency-selection latency;
- thermal headroom;
- background-task caps.

A scalar priority cannot encode every objective; production schedulers combine class, weight, utilization, topology, and QoS signals.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification**
- 9 Synthesis

Observing Linux scheduling attributes

```
1 # Niceness affects weight in the normal/fair class
2 nice -n 10 ./cpu_bound
3 renice 5 -p $PID
4
5 # Inspect policy, priority, affinity, and runtime fields
6 chrt -p $PID
7 taskset -pc $PID
8 ps -o pid,cls,rtprio,pri,ni,psr,stat,comm -p $PID
9
10 # Real-time policy changes require privilege and care
11 sudo chrt --fifo 20 ./short_bounded_task
```

Safety

Never test unbounded real-time CPU loops on a production machine. Use a VM, retain a rescue shell on another CPU, and enforce a runtime limit.

Priority inheritance with POSIX mutexes

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 static void check(int error) {
6     if (error == 0) return;
7     fprintf(stderr, "%s\n", strerror(error));
8     exit(EXIT_FAILURE);
9 }
10 int main(void) {
11     pthread_mutexattr_t attributes;
12     pthread_mutex_t mutex;
13     check(pthread_mutexattr_init(&attributes));
14     check(pthread_mutexattr_setprotocol(&attributes,
15                                         PTHREAD_PRIO_INHERIT));
16     check(pthread_mutex_init(&mutex, &attributes));
17     check(pthread_mutexattr_destroy(&attributes));
18     check(pthread_mutex_destroy(&mutex));
19 }
```

Configuration smoke test, not a contention benchmark. Build with `cc -std=c11 -Wall -Wextra -pthread`. Pthread functions return error numbers directly; report those, not `errno`.

Event-driven queue scheduler skeleton (pseudocode)

```
1 while unfinished:
2     expired = settle_completion_and_budgets(now)
3     enqueue_arrivals(now)
4     enqueue_expired_or_demoted(expired)
5     apply_due_aging_or_global_boost(now)
6     preserve_and_preempt_if_higher_queue_ready()
7     running = retain_current_or_select_head()
8     if running is None:
9         now = next_arrival_time() # record the idle interval
10        continue
11    event = min(next_arrival, completion, quantum_end,
12               allotment_end, boost_time, aging_time)
13    execute_and_charge_all_budgets(running, event - now)
14    now = event
```

For MLFQ, store level, remaining quantum, remaining allotment, first start, and last boost epoch explicitly.

Verification checklist

For every computed schedule, verify:

1. no task executes before arrival or while blocked;
2. total executed service equals each b_i ;
3. the selected task is maximal under the complete policy and tie rule;
4. preemption occurs only at defined events;
5. priority inheritance/boosts are restored correctly;
6. MLFQ quantum and allotment accounting remain consistent across yields; and
7. context-switch and idle intervals are included when nonzero.

Road map

- 1 From Week 5 to Progress Guarantees
- 2 Starvation and Bounded Service
- 3 Priority Inversion and Resource Protocols
- 4 Multilevel Queue Scheduling
- 5 Multilevel Feedback Queue Scheduling
- 6 Real-Time Priority Analysis
- 7 Modern Scheduler Architectures
- 8 Implementation and Verification
- 9 **Synthesis**

Misconceptions corrected

Misconception	Precise statement
Lower numeric priority always means higher urgency. Aging automatically prevents starvation.	$\Rightarrow$ Numeric convention is API/policy specific. $\Rightarrow$ It needs sufficient, eventually dominant boosting and fair ties.
MLFQ always promotes a task that blocks early. Demotion alone prevents starvation.	$\Rightarrow$ Promotion and accounting are design parameters. $\Rightarrow$ Low queues may still starve without boosts or shares.
Priority inheritance makes H run immediately. A real-time priority guarantees a deadline.	$\Rightarrow$ It accelerates the resource owner; H remains blocked. $\Rightarrow$ Guarantees require WCET, arrival, blocking, and over-head analysis.
Linux fair scheduling is just classic CFS. Containers each have their own CPU scheduler.	$\Rightarrow$ The fair class is transitioning to EEVDF concepts. $\Rightarrow$ Host-kernel scheduling is shaped by cgroup policy.

Policy selection by requirement

Requirement	Likely mechanism	Question to ask
critical control latency	fixed priority/deadline class	is demand admitted and blocking bounded?
interactive responsiveness	feedback/latency-aware fair policy	can behavior be gamed?
tenant isolation	hierarchical shares and quotas	what happens to unused capacity?
background progress	aging, boosts, reserved share	is there a finite service bound?
lock-dependent real time	inheritance/ceiling protocol	are critical sections bounded?
mobile efficiency	topology/energy-aware placement	what latency and thermal signals are used?

Key takeaways

- Ready waiting, dependency blocking, and retry failures need different remedies.
- Starvation is a liveness property; aging helps only under stated assumptions.
- Priority inversion arises through resource ownership; inheritance and ceilings bound its effects differently.
- MLQ provides static class separation; MLFQ predicts behavior through feedback.
- MLFQ correctness depends on cumulative accounting, boosts, and fully specified parameters.
- Real-time guarantees require response-time or schedulability analysis, not merely high priority.
- Modern kernels combine scheduling classes, hierarchical controls, topology, energy, and QoS.

Next week: real-time and proportional-share scheduling

- Rate-monotonic and deadline-monotonic priority assignment;
- Earliest Deadline First and processor-demand analysis;
- reservations, budgets, and sporadic servers;
- lottery and stride scheduling;
- overload behavior and admission control;
- multiprocessor real-time complications.

Preparation

Review ceilings, recurrence relations, utilization, and the difference between sufficient and exact schedulability tests.

Road map

10 Appendix

Knowledge check: progress and dependencies

1. Why can a mutually exclusive program still deadlock or starve?
2. Construct a workload that indefinitely denies a runnable job service.
3. Why does reaching the highest aging rank not by itself bound waiting?
4. How does priority donation propagate when the owner is also blocked?
5. Why must unlock recompute priority instead of always restoring base rank?
6. Distinguish ordinary resource blocking from unrelated medium-priority interference.

Exercise: preemptive priority with aging

Jobs (a, b, π) : $A = (0, 7, 1)$, $B = (1, 3, 3)$, $C = (2, 5, 2)$, $D = (3, 2, 4)$. Larger π is higher; ties are FIFO.

1. Use Week 5's fixed-priority schedule as a baseline and record maximum wait.
2. Specify when waiting credit resets and how equal effective ranks are ordered.
3. Repeat with effective priority increased by one after every four units of ready waiting.
4. State exactly when boosts occur and whether a boost can trigger immediate preemption.
5. Compare mean latency and maximum waiting.

Exercise: diagnose priority inversion

Tasks L , M , and H have increasing priority. L locks R for 3 ms; after 1 ms, H becomes ready and requests R ; then M becomes ready for 8 ms.

1. Draw the schedule without a locking protocol.
2. Measure H 's direct and indirect blocking.
3. Redraw with priority inheritance.
4. What changes if L blocks while holding R ?
5. Could inheritance prevent a circular-wait deadlock? Explain.

Exercise: specify and trace an MLFQ

Configure $Q_0 : q = 2$, $Q_1 : q = 4$, $Q_2 : q = 8$, with allotments $(4, 8, \infty)$ and a global boost every 20 units. Jobs: $A = (0, 14)$, $B = (1, 2)$, $C = (7, 5)$.

1. State the arrival/quantum-boundary event order.
2. Use the lecture's boost merge order and interrupted-slice preservation; draw the timeline.
3. Track remaining service, quantum, and allotment separately.
4. Compute response and turnaround times.
5. Modify A to yield every 1.9 units. Does cumulative accounting stop gaming?

Exercise: response-time analysis

Fixed priorities follow increasing period:

$$\tau_1 = (C = 1, T = 4, D = 4), \quad \tau_2 = (1, 5, 5), \quad \tau_3 = (2, 10, 10).$$

Assume $B_1 = 0$, $B_2 = 0$, and $B_3 = 1$.

1. Compute utilization. Why is the plain no-blocking utilization bound insufficient here?
2. Compute each fixed-point response time.
3. Is every task schedulable under the model?
4. Increase B_3 until its deadline is first missed.
5. Explain why utilization alone cannot represent blocking.

Further study

- fixed-priority response-time analysis with jitter and self-suspension;
- priority inheritance, original ceiling, and immediate-ceiling protocols;
- BSD and historical Unix feedback schedulers;
- Linux EEVDF, deadline scheduling, cgroup CPU control, and utilization clamping;
- Windows base priorities, boosts, QoS, and heterogeneous-core placement;
- scheduler activations and user-level runtimes;
- adversarial workloads and scheduler side channels.

Questions?

Source notes: theory and protocol contracts

- R. and A. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 8: MLFQ, allotment accounting, and boosts.
<https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf>
- L. Sha, R. Rajkumar, J. P. Lehoczky (1990), *Priority Inheritance Protocols: An Approach to Real-Time Synchronization*. IEEE Transactions on Computers 39(9), 1175–1185. DOI: 10.1109/12.57058.
- M. Joseph and P. Pandya (1986), *Finding Response Times in a Real-Time System*. The Computer Journal 29(5), 390–395. DOI: 10.1093/comjnl/29.5.390.
- The Open Group, `pthread_mutexattr_setprotocol`: PI and ceiling attributes.
<https://pubs.opengroup.org/onlinepubs/9799919799/>

Numerical schedules and bounds in this deck are teaching examples with explicitly stated assumptions, not production-kernel traces.

Source notes: implementation case studies

Official documentation checked September 16, 2026:

- Linux rt-mutex: inheritance, chained donation, and waiter updates.
<https://docs.kernel.org/locking/rt-mutex.html>
- Linux EEVDF: eligibility/lag and virtual-deadline selection.
<https://docs.kernel.org/scheduler/sched-eevdf.html>
- Linux cgroup v2: CPU weights, bandwidth limits, and resource control.
<https://docs.kernel.org/admin-guide/cgroup-v2.html>
- Microsoft Learn: Scheduling Priorities.
<https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities>

Implementation behavior depends on OS version and configuration. Virtual deadlines in a fair scheduler are not application deadline guarantees.